

TRADER TRACKER

**Projet réalisé dans la cadre du passage du Titre
Professionnel de**

Développeur Web et Web Mobile

Niveau 5

Présenté par Julien Alaria

SOMMAIRE

1- Introduction

2- Compétences du référentiel couvertes par le projet

2.1 Activité type n°1 : Développer la partie front-end d'une application web ou web mobile sécurisée

- 1- Installer et configurer son environnement de travail en fonction du projet web ou web mobile
- 2- Maquetter des interfaces utilisateur
- 3- Réaliser des interfaces utilisateur statiques
- 4- Développer la partie dynamique des interfaces utilisateur

2.2 Activité type n°2 : Développer la partie back-end d'une application web ou web mobile sécurisée

- 5- Mettre en place une base de données relationnelle
- 6- Développer des composants d'accès aux données SQL et NoSQL
- 7- Développer des composants métier coté serveur
- 8- Documenter le déploiement d'une application dynamique web ou web mobile

3- Fondation Conceptuelle et direction du projet

- 3.1 Présentation générale
- 3.2 Besoins et objectifs de l'application
- 3.3 Users Stories
- 3.4 MVP
- 3.5 Positionnement stratégique
- 3.6 Cahier des charges

4- Environnement et Architecture technique

- 4.1 Environnement de développement
- 4.2 Choix techniques
- 4.3 Architecture applicative
- 4.4 Versioning & workflow Git

5- Conception du Front-end

- 5.1 Arborescence
- 5.2 Wireframes
- 5.3 Charte Graphique
- 5.4 Maquettes

6- Conception du Back-End

- 6.1 Base de données

6.2 Dictionnaire de données

7- Implémentation du Front-End

7.1 Architecture et logique d’affichage

7.2 Esthétique, accessibilité et performances

7.3 Sécurité et robustesse du front

8- Implémentation du Back-End

8.1 Architecture et logique métier

8.2- Recommandations et autorisation par spécialisation

8.3- Sécurité et intégrité des données

9- Routes

9.1- Architecture API

10- Sécurité de l’application

10.1 Système d’authentification

10.2 Gestion de sessions, rôles et middlewares

11- Accessibilité

11.1 Seo

11.2 Light house

11.3 Rgpd

12- Tests et jeu d’essai

12.1 Vérifications manuelles

12.2 Tests automatisés

12.3 jeu d’essai

13- Déploiement de l’application

13.1 Préparation de l’environnement de production

13.2 Configuration

13.3 Mise en ligne et exploitation

14- Roadmap et évolutions futures

14.1- Planning Prévisionnel

14.2- Évolutions possibles

15- Conclusion

Annexe

Stack technique et modules applicatifs

1- INTRODUCTION

Ce dossier présente Trader Tracker, projet fictif réalisé dans le cadre de la Formation CDPI 2026 à l'école du numérique La Plateforme de Cannes pour l'obtention du Titre Professionnel Développeur Web et Web Mobile.

Il retrace l'ensemble de la démarche, de la conception à la mise en production : analyse du besoin, choix techniques, modélisation de la base de données, implémentation du front-end et du back-end, sécurisation de l'application et déploiement.

Trader Tracker est une plateforme web permettant à des investisseurs particuliers de suivre des actifs financiers (actions, devises, matières premières) et de consulter les recommandations publiées par des analystes spécialisés.

Le projet couvre l'ensemble des compétences attendues sur les deux activités du référentiel : développement front-end et back-end d'une application web sécurisée.

Ce document détaille les choix d'architecture, les modèles de données, l'implémentation des fonctionnalités par profil utilisateur (visiteur, utilisateur, analyste, administrateur) ainsi que les mécanismes de sécurité mis en place pour protéger l'application et ses utilisateurs.

2- COMPETENCES DU REFERENTIEL

Le projet Trader Tracker a été conçu pour répondre aux exigences du Titre Professionnel Développeur Web et Web Mobile, en couvrant l'ensemble des compétences techniques attendues dans les deux grandes activités du référentiel : le développement Front-End et le développement Back-End, intégrant les notions essentielles de sécurité, d'accessibilité, de modélisation de données, d'interactivité et de contenu dynamique.

2.1 Activité type n°1 : Développer la partie front-end d'une application web ou web mobile sécurisée

1- Installer et configurer son environnement de travail en fonction du projet web ou web mobile sécurisé

Développement sous les OS Linux Debian et Microsoft Windows 11 avec Visual Studio Code comme IDE.

Back-end avec NodeJs et Front-end sous JavaScript ES6 modules.

Outils de développement nécessaires :

- des outils de gestion de version (GitHub)
- conteneurs implémentant les services requis pour l'environnement de développement
- documentation technique de l'environnement de travail
- Mise en place d'un serveur de données local
- utilitaire de contrôle de qualité de code

2- Maquetter des interfaces utilisateur

Web design avec utilisation de Figma et une approche mobile-first.

Interface simple, intuitive et ergonomique avec utilisation de composants réutilisables.

3- Réaliser des interfaces utilisateur statiques

Interface conforme à la maquette avec sémantique HTML, CSS.

Contrastes de couleurs et focus visible pour l'accessibilité, le référencement SEO (Search Engine Optimisation) en connaissance des standards de l'organisme de normalisation W3C (World Wide Web Consortium) et ECMAScript.

Design responsive pour adaptation de l'interface au type de support desktop et mobile.

4- Développer la partie dynamique des interfaces utilisateur

Le développement de la partie dynamique des interfaces utilisateur, repose essentiellement sur l'utilisation de JavaScript et de son écosystème.

Interface fluidifiée grâce à l'utilisation de mécanismes asynchrones (AJAX) avec validation systématique des entrées, gestion des erreurs et exceptions pour éviter les vulnérabilités.

Utilisation d'un service d'API local pour les données relatives aux produits financiers sous format de données JSON (Javascript Object Notation).

Les données financières (référencement des actifs et historique des cours) sont collectées une fois en amont auprès de l'API externe Massive (*anciennement Polygon.io*) par des scripts dédiés , puis stockées localement et servies depuis ces fichiers, sans appel à l'API externe au moment de la consultation par l'utilisateur.

2.2 Activité type n°2 : Développer la partie back-end d'une application web ou web mobile sécurisée

5- Mettre en place une base de données relationnelle

Utilisation de la méthode MERISE, ajout d'un fichier README avec le script SQL pour la création de la base de donnée ou restauration en cas d'incident et de seeders pour insertion de données et d'utilisateurs avec leurs droits d'accès.

6- Développer des composants d'accès aux données SQL et NoSQL

Utilisation de mysql2 sous NodeJs, très populaire, rapide et moderne.

Utilisation de requêtes préparées contre les injections SQL, pool de connexions pour optimiser les interactions avec la base de données.

7- Développer des composants métier coté serveur

Les composants métier sécurisés côté serveur gèrent la logique principale d'une application backend. ils sont souvent développés avec Express pour améliorer la maintenabilité, la sécurité et l'organisation du code serveur.

8- Documenter le déploiement d'une application dynamique web ou web mobile

La documentation intègre un fichier README.md (Markdown) qui comprend la procédure de déploiement de l'application (installation du projet, configuration des variables d'environnement (fichier .env.example), lancement du serveur et utilisation de la base de données) et scripts de déploiement.

3- Fondation conceptuelle et direction du projet

1- Présentation générale

Inspirée des plateformes de réseaux sociaux et des sites de financiers Trader Tracker est une plateforme web dont l'objectif principal consiste à :

- consulter des données financières (matières premières, devises, actions) ;
- suivre l'évolution des prix ;
- suivre des analystes financiers spécialisés et consulter leurs recommandations.

2- Besoins et objectifs de l'application

L'application vise à répondre au besoin suivant :

- Permettre à des investisseurs particuliers de suivre facilement des actifs financiers et de bénéficier d'analyses afin d'aider à la prise de décision ;
- Au lieu de devoir glaner les informations sur plusieurs plateformes (courtiers, réseaux sociaux ou sites de streaming), l'utilisateur retrouvera toutes ces informations sur l'application ;
- Un visiteur découvre librement le contenu public de l'application (produits financiers, recommandations et analystes) sans contrainte ;
- S'il souhaite interagir davantage, il sera orienté vers la création d'un compte en tant que utilisateur inscrit ou analyste financier (qui devra être approuvé par l'administration de l'application) ;
- Une fois identifié un utilisateur inscrit pourra suivre des actifs financiers et des analystes ;
- Un compte analyste pourra également commenter les actifs financiers correspondant à la spécialité définie lors de la création du compte ;
- L'administrateur gère l'application depuis un panel simple et intuitif ou il pourra approuver les analystes et éditer les comptes utilisateurs et recommandations analystes.

3- Users Stories

Public

En tant que public je veux pouvoir m'inscrire

En tant que visiteur je veux consulter les pages détails d'actifs et d'analystes

Utilisateur

En tant que utilisateur je veux me connecter à mon compte afin d'accéder à mes suivis d'actifs (prix) et d'analystes (recommandations)

En tant que utilisateur je veux pouvoir modifier mes données personnelles afin de mettre mon profil à jour

En tant que utilisateur je veux consulter les pages détails d'actifs et d'analystes

Analyste

En tant qu'analyste je veux créer un compte afin de pouvoir accéder à mon espace personnalisé

En tant qu'analyste je veux pouvoir modifier mes données personnelles afin de mettre mon profil à jour

En tant qu'analyste je veux consulter les pages détails d'actifs et d'analystes

En tant qu'analyste je veux commenter et recommander (Buy / Sell / Hold) des actifs

Administrateur

En tant qu'administrateur je veux consulter les pages détails d'actifs et d'analystes

En tant qu'administrateur je veux pouvoir consulter et éditer les différents profils

En tant qu'administrateur je veux pouvoir consulter et éditer les commentaires et recommandations des analystes

En tant qu'administrateur je veux pouvoir valider une demande de création de compte analyste

3.4 MVP (Minimum Viable Product)

Le périmètre fonctionnel retenu pour la version initiale est :

- authentification utilisateur sécurisée ;
- consultation des produits financiers via API Locale ;
- gestion des favoris ;
- publication de recommandations par des analystes ;
- interface d'administration.

Les fonctionnalités avancées (notifications en temps réel via WebSockets, OAuth, alertes de prix, gestion d'API, etc.) seront traitées en extension si le temps le permet.

3.5 Positionnement stratégique

Trader Tracker se positionne comme une plateforme simplifiée de suivi d'actifs financiers destinée aux investisseurs particuliers souhaitant centraliser leurs suivis et analyses dans une interface intuitive et accessible.

Les différentes plateformes disponibles (courtiers, réseaux sociaux et sites de streaming) ne proposant pas de solution simplifiée et optimisée.

3.6 Cahier des charges

Le cahier des charges fonctionnel fixe le périmètre de l'application autour de quatre profils d'utilisateurs aux droits distincts : visiteur, utilisateur inscrit, analyste et administrateur.

La conception du projet est planifiée sur 13 semaines (du 13 avril au 13 juillet).

Le projet à été structuré sur la base d'un MVP (Minimum Viable Product) pour un ensemble cohérent, fonctionnel et réalisable dans le temps imparti.

Cette version comprend un un système d'inscription authentification, un système d'édition de recommandations, une landing page, une page détail pour les produits et analystes financiers, une page proposant tous les actifs proposés sur l'application répertoriés sous forme de liste, une page dédiée pour les rôles utilisateur et analyste, une panel admin ainsi qu'une API locale pour les informations relatives aux produits financiers.

Cette base étant sujette à de futurs innovations et évolutions.

Exigences fonctionnelles

Le visiteur peut consulter librement le catalogue d'actifs financiers (actions, devises, matières premières), leurs détails et les recommandations publiques des analystes, puis créer un compte.

L'utilisateur inscrit peut suivre des actifs et des analystes, consulter ses suivis dans un espace dédié, et mettre à jour ses informations personnelles.

L'analyste, après validation de son compte par un administrateur, peut publier, modifier et supprimer des recommandations (BUY / SELL / HOLD) sur les actifs correspondant à sa spécialisation.

L'administrateur dispose d'un espace de gestion lui permettant de consulter et modifier les comptes utilisateurs, de valider les demandes d'inscription analyste, et de superviser les recommandations publiées et publier des recommandations .

Exigences techniques

Application en Single Page Application (SPA), responsive mobile-first, consommant une API REST sécurisée par JWT.

Les données financières (référencement des actifs et historique des cours) sont collectées une fois en amont auprès de l'API externe Massive (*anciennement Polygon.io*), via des scripts dédiés, puis stockées localement dans des fichiers JSON et servies depuis ceux-ci ; la base de données ne conserve que le référencement des actifs (nom, ticker, type).

Les mots de passe sont hachés (bcrypt) et jamais stockés en clair ; les routes sensibles sont protégées par un middleware de rôle vérifié côté serveur et de spécialisation pour les recommandations.

Exigences non-fonctionnelles

Accessibilité et SEO : sémantique HTML5, contrastes suffisants, structure de titres cohérente.

Performance : chargement progressif des listes d'actifs (pagination) et en lazyloading pour les carrousels, mise en cache locale du token de session.

Sécurité : validation des entrées côté client et serveur, protection CORS, variables sensibles isolées dans un fichier .env non versionné, gestion des limitations de requêtes répétées via express-rate-limit et configuration des en-têtes.

4 Environnement et Architecture technique

4.1 Environnement et développement

Développement sous les OS Linux Debian et Microsoft Windows 11 avec Visual Studio Code comme IDE.

Back-end avec NodeJs et Front-end sous JavaScript ES6 modules.

4.2 Choix techniques

Front-end : JavaScript ES6 modules,

 HTML 5,

 CSS 3,

 Apex charts ,

 Prettier

Back-end : Node.js 24.11.1,

 Express.js 5.2.1,

 JWT 9.0.3,

 bcrypt 6,0,0,

 Massive Api 10.7.1 (*anciennement Polygon.io*),

 cors 2.8.6,

 express-rate-limit 8.5.2,

 multer 2.2.0,

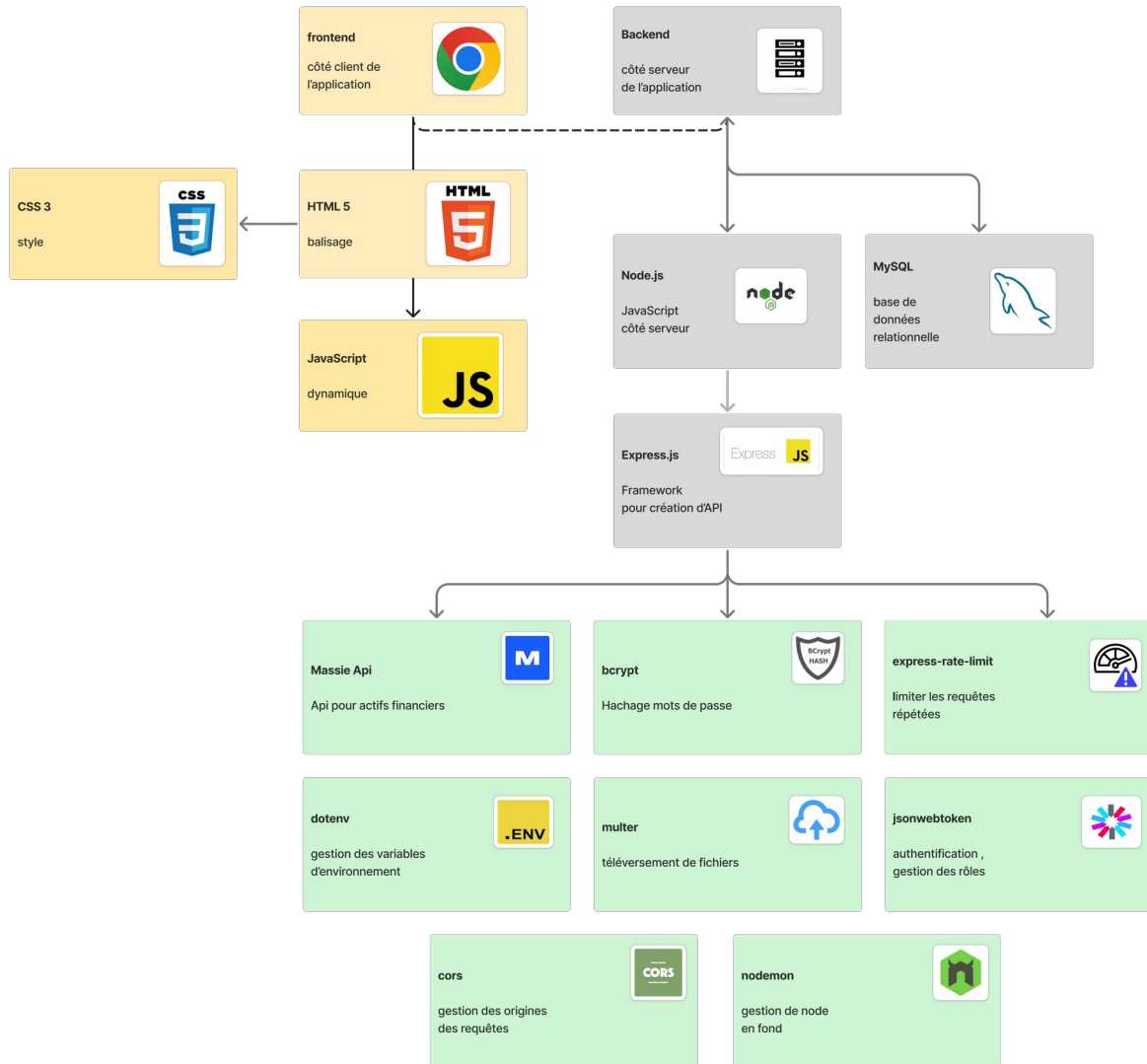
 mysql2 3.22.3,

 nodemon 3.1.14

Base de données : MySQL 8.0.46 sous Windows 11 et MariaDB 11.8.6 sous debian-linux-gnu (x86_64)

J'ai volontairement implémenté les utilitaires pour ce projet, dans un but pédagogique.

SCHEMA DE LA STACK



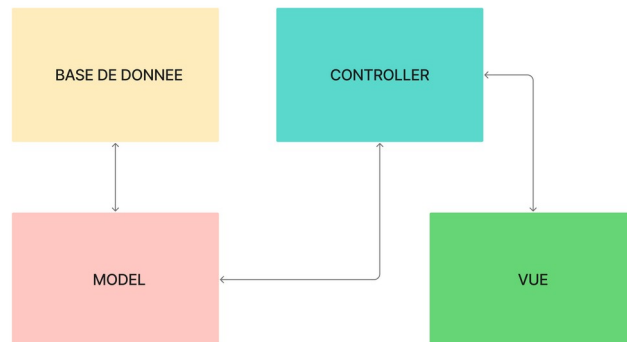
4.3 Architecture applicative

Application fullstack basée sur une séparation : Front-end (client) Back-end (serveur)

Base de données

Architecture de type MVC (Model – View – Controller).

Séparation des responsabilités, code réutilisable, facilité de maintenance et évolutivité.



4.4 Versioning et workflow Git

Projet versionné en continu sur un dépôt GitHub, organisation simple avec une branch main toujours stable et déployable, et des branches dédiées à chaque fonctionnalités qui seront fusionnées sur main après vérification.

Commits fréquents et explicites.

Ajout d'un fichier .gitignore pour n'envoyer que les fichiers utiles et éviter la surcharge sur le remote et protéger les secrets.

Le fichier .env n'est jamais versionné et .env.example sert de référence pour les autres développeurs.

Utilisation de GitHub Projects en model Kanban (A faire / En cours / Fait) avec tickets liés aux branches.



5 Conception du Front

5.1 Arborescence

L'arborescence repose sur une structure logique et épurée :

- Accueil : introduction, barre de recherche, présentation des produits et analystes financiers proposés sous forme de carrousels et lien vers la liste globale des actifs proposés ;
- Liste des actifs : liste de tous les actifs disponibles sur le site ;
- Détail d'actif : affichage du nom, logo, informations générales, bouton de suivi, graphique de l'évolution des prix sur une durée de un an, recommandations et analystes relatifs à ce produit avec un formulaire pour édition de recommandation selon le rôle (analyst ou admin) ;
- détail d'analyste : présentation de l'analyste, bouton de suivi et ses recommandations ;
- Espace utilisateur : informations de l'utilisateur avec photo de profil, watchlists des produits et analystes suivis sous forme de carrousel et de listes ainsi qu'un formulaire d'edition de son propre profil ;
- Espace analyste : informations sur l'analyste avec photo de profil, watchlists des produits et analystes suivis sous forme de carrousel et de listes, edition ses propres recommandations ainsi qu'un formulaire d'edition de son propre profil ;
- Espace admin : Possibilité d'approuver les analystes, éditer les recommandations des analystes, publier des recommandations et gestion des utilisateurs et commentaires.

5.2 Wireframes



Wireframe réalisé sur Figma en amont de la création de la maquette.

L'ensemble des écrans clés a été modélisé en basse fidélité avant la maquette :

pages publiques (accueil, détail d'un actif), parcours d'authentification (connexion, inscription utilisateur et analyste) et espaces privés (utilisateur, analyste, administrateur).

Cette étape a permis de valider l'organisation des informations et la navigation avant tout travail graphique.

	#1d3a8f color-navy-100 navbar		#e5ecef color-bg-page general background (admin)
	#102a72 color-navy-200 hero		#f8f9fa color-bg-soft light backgrounds (neutral buttons)
	#081f5c color-navy-300 middle		#3f4f6 color-bg-hover row hover
	#03034Bf color-navy-400 buttons, links		#f9f9f9 color-bg-empty placeholders / empty states
	#020233 color-navy-500 usage		#cccccc color-border Borders / separators
	FFA500 orange hover		#e3e6ec color-border-light Borders / separators
	#000000 color-black borders, bold text		#e9ecef color-border-soft Borders / separators
	#1a1a1a color-dark dark backgrounds nav-register, logout		#b8daff pagin-button-hover Borders / separators
	#ffffff color-white		#ff0000 color-danger delete button
	#333333 color-text body text		#dbe02c color-success edit/confirm button
	#555555 color-text-muted secondary text		#a3a815 color-success-border edit/confirm button
	#777777 color-text-light subtle text		#1a73e8 color-info links/info
	#888888 color-text-faint very subtle text		#F80505 focus-visib accessibility

5.3 Charte Graphique

texte courant sans-sérif pour la lisibilité et référencement SEO.

La charte repose sur un bleu marine comme couleur de marque, déclinée sur la navbar et le footer, complétée par un vert et un rouge pour les signaux de recommandation (BUY / SELL), un orange pour les accents et appels à l'action, et un jaune pour les actions de modification côté administration.

Le blanc, le noir et le gris structurent le texte et les fonds neutres.

Cette palette garantit des contrastes suffisants pour l'accessibilité tout en restant sobre et cohérente avec l'univers financier.

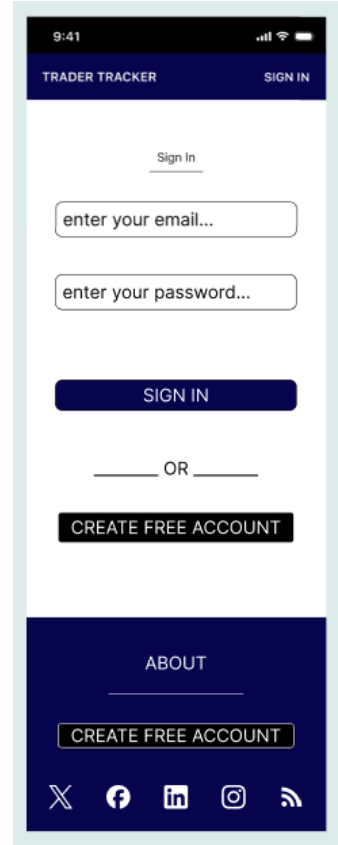
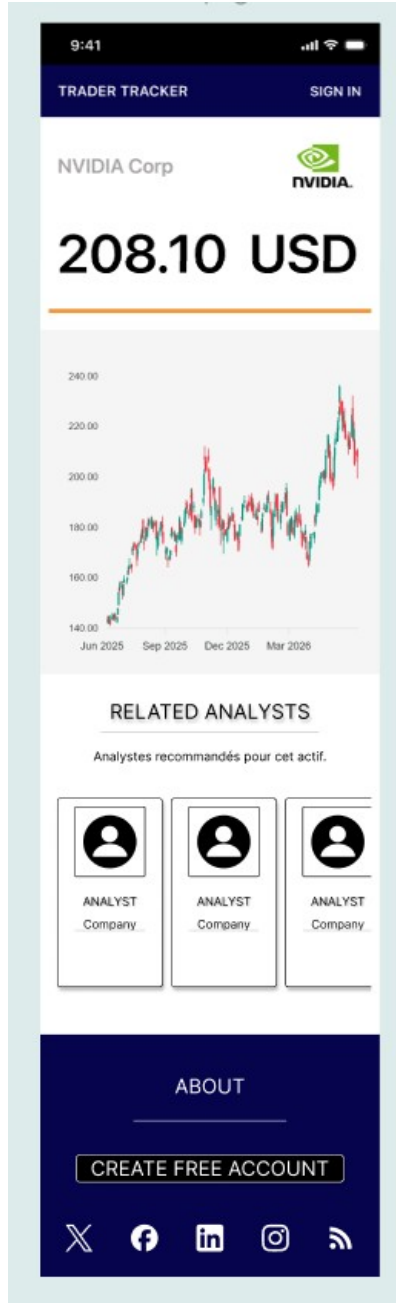
5.4 Maquettes

Les maquettes ont été réalisées sur Figma premièrement en version mobile first puis adaptée pour desktop.

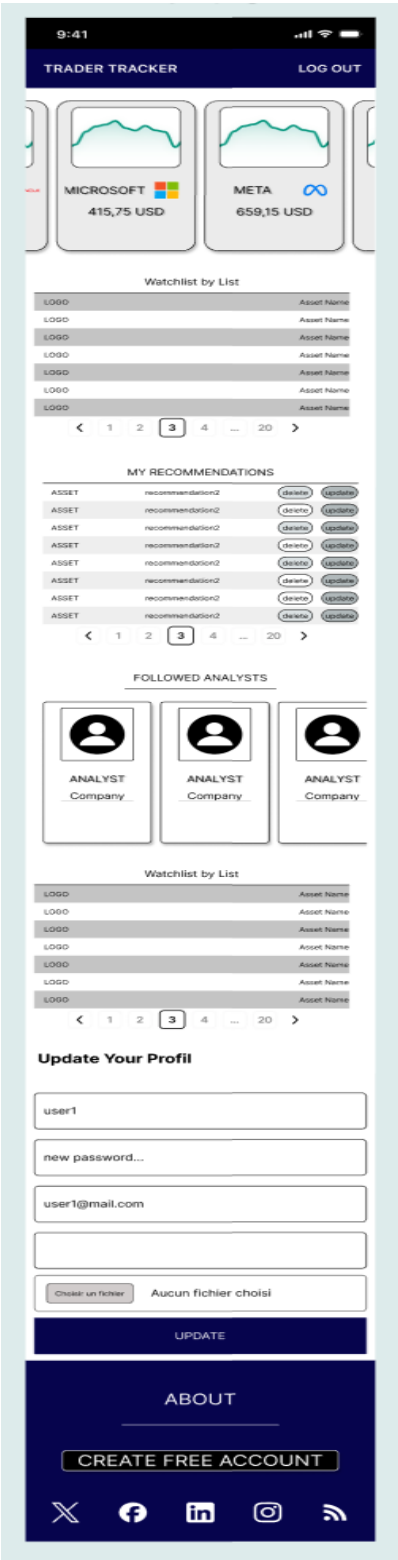
Les écrans présentés ci-dessus illustrent le parcours principal :

la page d'accueil avec ses sections par marché (Nasdaq, Forex, Comex) et ses analystes mis en avant, la page de détail d'un actif avec son graphique de cours et les analystes associés, le formulaire de connexion, ainsi que l'espace analyst connecté regroupant la watchlist en carrousel et paginée, ses propres recommandations pouvant être éditées, les analystes suivis et le formulaire de mise à jour du profil.

La maquette desktop reprend cette même logique de présentation.



Utilisation de composants



6- CONCEPTION DU BACKEND

6.1 Base de données

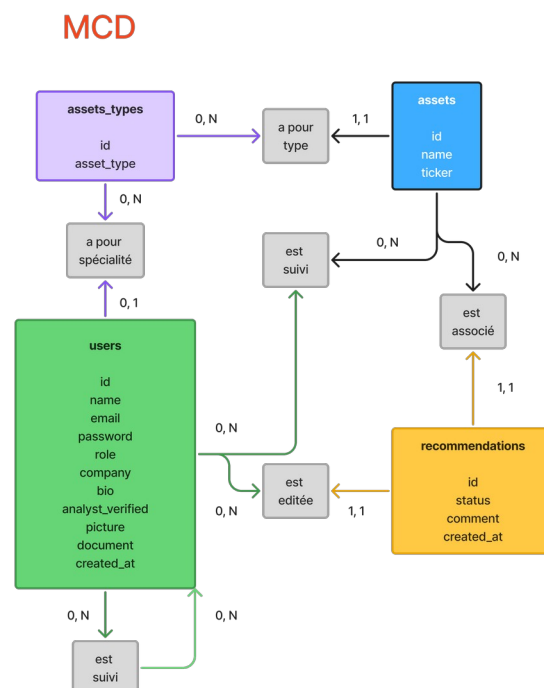
Modèle relationnel pensé autour de plusieurs entités centrales :

- users : utilisateur inscrit, analyste et administrateur ;
- assets : les différents actifs financiers (seulement les données immutables seront insérées en base de donnée) ;
- assets-types : les types d'actifs financiers ;
- recommendations : recommandations des analystes ;
- users_assets_follow : les actifs suivis par les utilisateurs.

Utilisation de la méthode MERISE :

- 1- modèle conceptuel de données (MCD) avec identification des entités et leurs associations ;
- 2- modèle logique de données (MLD) avec transformation des entités en tables, ajout des clés, et des associations N-N en tables pivots ;
- 3- modèle physique de données (MPD) avec mise en place des tables, colonnes et contraintes selon la syntaxe du système de gestion de base de données utilisé.

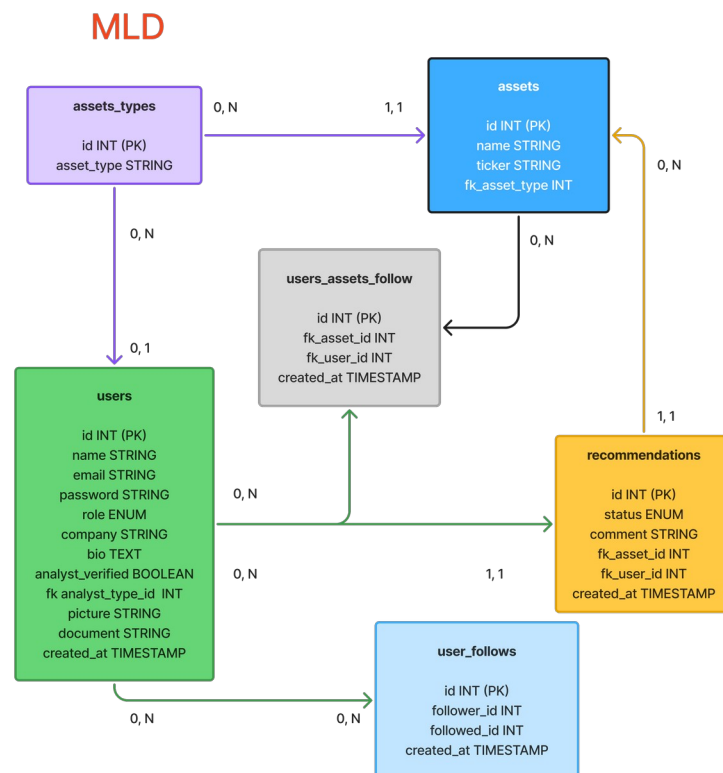
Le MCD repose sur les notions d'entité, d'association et de relations.



La transcription d'un MCD en MLD (Modèle Logique de Données) s'effectue selon quelques règles simples qui consistent d'abord à transformer toute entité en table, avec l'identifiant comme clé primaire, puis à observer les valeurs prises par les cardinalités maximum de chaque association pour représenter celle-ci

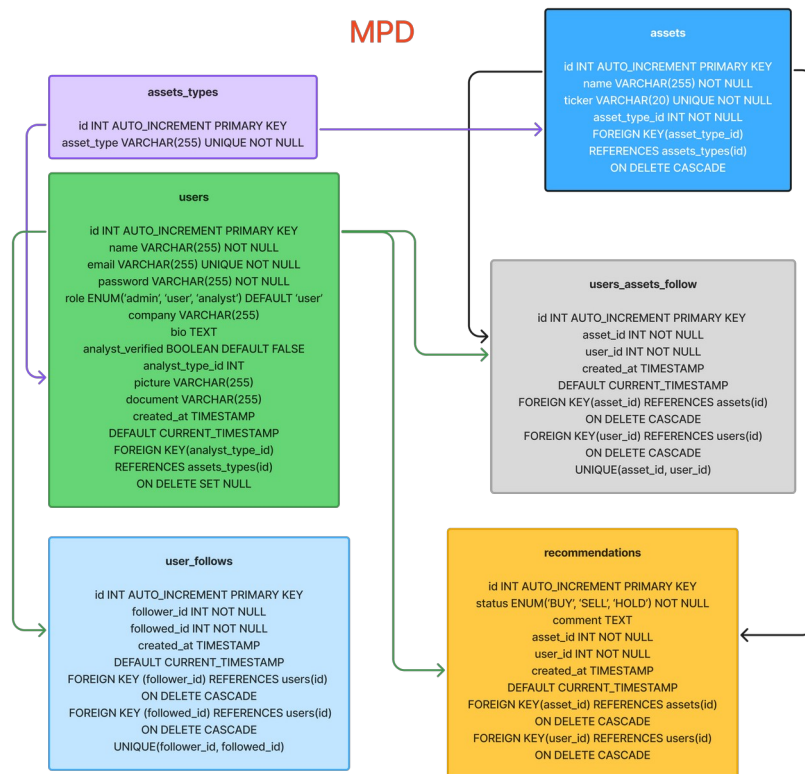
La règle des cardinalités :

- 1-1 : création d'un nouveau champ dans la table
- 1-N : la clé primaire de N devient une clé étrangère dans 1
- N-N : création d'une table pivot



Dans la méthode Merise, le MPD (Modèle Physique de Données) correspond à l'implantation concrète de la base de données.

Dans le cadre de ce projet, j'utilise le SGBDR (Système de Gestion de Base de Données Relationnelle) MySQL, qui utilise le langage SQL.



```

CREATE DATABASE IF NOT EXISTS trader_tracker;
USE trader_tracker;

CREATE TABLE assets_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  asset_type VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE assets (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  ticker VARCHAR(20) UNIQUE NOT NULL,
  asset_type_id INT NOT NULL,

  FOREIGN KEY (asset_type_id)
  REFERENCES assets_types(id)
  ON DELETE CASCADE
);

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'user', 'analyst') DEFAULT 'user',
  company VARCHAR(255),
  bio TEXT,
  analyst_verified BOOLEAN DEFAULT FALSE,
  analyst_type_id INT,
  picture VARCHAR(255),
  document VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (analyst_type_id)
  REFERENCES assets_types(id)
  ON DELETE SET NULL
);

CREATE TABLE users_assets_follow (
  id INT AUTO_INCREMENT PRIMARY KEY,
  asset_id INT NOT NULL,
  user_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(asset_id, user_id),

  FOREIGN KEY (asset_id)
  REFERENCES assets(id)
  ON DELETE CASCADE,

  FOREIGN KEY (user_id)
  REFERENCES users(id)
  ON DELETE CASCADE
);

CREATE TABLE recommendations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  status ENUM('BUY', 'SELL', 'HOLD') NOT NULL,
  comment TEXT,
  asset_id INT NOT NULL,
  user_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (asset_id)
  REFERENCES assets(id)
  ON DELETE CASCADE,

  FOREIGN KEY (user_id)
  REFERENCES users(id)
  ON DELETE CASCADE
);

CREATE TABLE user_follows (
  id INT AUTO_INCREMENT PRIMARY KEY,
  follower_id INT NOT NULL,
  followed_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY unique_follow (follower_id, followed_id),
  FOREIGN KEY (follower_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (followed_id) REFERENCES users(id) ON DELETE CASCADE
);

```

6.2 Dictionnaire des données

Il contient les informations sur les tables, colonnes, types de données, clés primaires, relations et contraintes.

Il sert à comprendre l'organisation de la base, faciliter le développement et la maintenance , documenter les données du projet et éviter les erreurs entre développeurs.

TABLE : ASSETS TYPES

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
asset_type	type de l'actif financier	VARCHAR	255	Oui

TABLE : ASSETS

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
name	nom de l'actif financier	VARCHAR	255	Oui
ticker	symbole boursier unique	VARCHAR	20	Oui
asset_type_id	clé étrangère de assets_types	INT	-	Oui

TABLE : USERS

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
name	nom de l'utilisateur	VARCHAR	255	Oui
email	email de l'utilisateur	VARCHAR	255	Oui
password	mot de passe utilisateur	VARCHAR	255	Oui
Role	rôle utilisateur	ENUM	-	Oui
company	société de l'analyste	VARCHAR	255	Non
bio	biographie de l'analyste	TEXT	-	Non
analyst_verified	Validation admin	BOOLEAN	-	Non
	pour analyste			
analyst_type_id	clé étrangère de assets_types	INT	-	Non
picture	photo de profil	VARCHAR	255	Non
document	certification de l'analyste	VARCHAR	255	Non
created_at	date de création de l'utilisateur	TIMESTAMP	-	Oui

TABLE : USERS ASSETS FOLLOW

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
asset_id	clé étrangère de assets	INT	-	Oui
user_id	clé étrangère de users	INT	-	Oui
created_at	date de création du suivi	TIMESTAMP		Oui

TABLE : RECOMMENDATIONS

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
status	status de la recommandation (BUY / SELL / HOLD)	ENUM	-	Oui
comment	commentaire sur la recommandation	TEXT	-	Non
asset_id	clé étrangère de assets	INT	-	Oui
user_id	clé étrangère de users	INT	-	Oui
created_at	date de création de la recommandation	TIMESTAMP	-	Oui

TABLE : USER FOLLOWS

Donnée	Description	Type	Taille	Obligatoire
id	identifiant unique	INT	-	Oui
follower_id	clé étrangère de users (celui qui suit)	INT	-	Oui
followed_id (celui suivi)	clé étrangère de users	INT	-	Oui
created_at	date de création du suivi	TIMESTAMP	-	Oui

7- IMPLEMENTATION DU FRONT-END

L'implémentation du frontend de ce projet repose sur les technologies suivantes :

Le projet est bâti avec JavaScript pour garantir une interactivité ;

Le style est assuré en CSS 3, utilisant des media queries pour assurer le responsive ;

L'arborescence organise les pages en répertoires distincts pour les accès publics et les accès authentifiés ou privés ;

Utilisation d'un wrapper pour éviter la duplication du code sur les fetch pour les requêtes HTTP vers le backend en définissant une baseURL commune.

7.1 Architecture et logique d'affichage

Router maison (Hash Router) pour gérer la navigation de l'application en Single Page Application (SPA).

L'application étant une SPA sans framework, la navigation ne repose pas sur des rechargements de page mais sur ce routeur fondé sur le hash de l'URL (#/details, #/admin, etc.).

À chaque changement de hash (événement hashchange) ou au chargement initial de la page (load), la fonction router() est déclenchée.

Elle lit le hash courant, puis importe dynamiquement (import()) le seul module de page correspondant à cette route.

Ce découpage en imports dynamiques évite de charger l'intégralité des pages de l'application dès le premier chargement.

Chaque page n'est récupérée par le navigateur qu'au moment où l'utilisateur y accède réellement.

Des Layouts sont utilisés pour englober les routes, permettant de partager des éléments globaux comme la barre de navigation.

Protection des routes par rôle (RoleGuard).

Pour les routes privées (/user, /analyst, /admin), le routeur appelle systématiquement la fonction roleGuard(roles autorisés) avant d'importer la page correspondante :

- Elle vérifie d'abord la présence d'un token JWT dans le localStorage ; en son absence, l'utilisateur est redirigé vers /login ;
- Elle décode ensuite le token pour en extraire le rôle, et vérifie qu'il fait partie des rôles autorisés pour cette route, dans le cas contraire, l'utilisateur est redirigé vers la page d'accueil ;
- Si l'une de ces deux vérifications échoue, le chargement de la page est interrompu avant même que le module correspondant ne soit importé.

Ce mécanisme, bien qu'indispensable pour l'expérience utilisateur (masquer les pages et menus non pertinents selon le rôle), reste une protection côté client et ne constitue jamais la sécurité réelle de l'application.

Un visiteur technique pourrait modifier manuellement son token ou appeler directement l'API sans passer par cette interface.

C'est pourquoi chaque route sensible de l'API backend est protégée indépendamment par un middleware serveur (AuthMiddleware), qui révérifie le rôle réel de l'utilisateur en base de données à chaque requête (voir section 10 - Sécurité de l'application).

RoleGuard et AuthMiddleware fonctionnent donc en duo : le premier pour l'ergonomie, le second pour la sécurité effective.

Extraits de front/router.js

```
import { roleGuard } from "../src/middlewares/roleGuard.js"
import { renderApp } from "../src/utils/layoutManager.js"

async function router() {
  // return to the top of page
  window.scrollTo(0, 0)

  const hash = (window.location.hash.slice(1) || "/").split("?")[0]

  let content = ""
  let init = null
  let layoutType = "default"

  switch (hash) {
    case "/":
      const home = await import("../src/pages/public/home.js")
      content = home.default
      init = home.initHome
      break

    case "/list":
      const list = await import("../src/pages/public/list.js")
      content = list.default
      init = list.initList
      break
```

```
    default:
      const notfound = await import("../src/pages/public/notfound.js")
      content = notfound.default
  }

  renderApp(content, layoutType)

  if (init) {
    try {
      // synchronous waiting for smooth rendering
      await init()
    } catch (err) {
      console.error("Error during initialization :", err);
    }
  }

  // from renderApp
  document.getElementById("root").style.opacity = "1"
}

window.addEventListener("hashchange", router)
// all dependencies loaded and ready to use, browser triggers the first display
window.addEventListener("load", router)
```

7.2 Esthétique, accessibilité et performances

semantique HTML & Audit LightHouse

Focus visible (navigation clavier)

Pour garantir l'accessibilité aux utilisateurs naviguant au clavier (touche Tab) plutôt qu'à la souris, l'application affiche un contour autour de tout élément interactif recevant le focus : liens, boutons, champs de formulaire et cartes cliquables.

Performance et qualité

responsive design en mobile-first

bonnes pratiques : KISS / DRY

code structuré et maintenable

tests fonctionnels

Un audit Lighthouse (Chrome DevTools) a été réalisé sur la page d'accueil de l'application :

Performance : 65 / 100

Accessibilité : 96 / 100

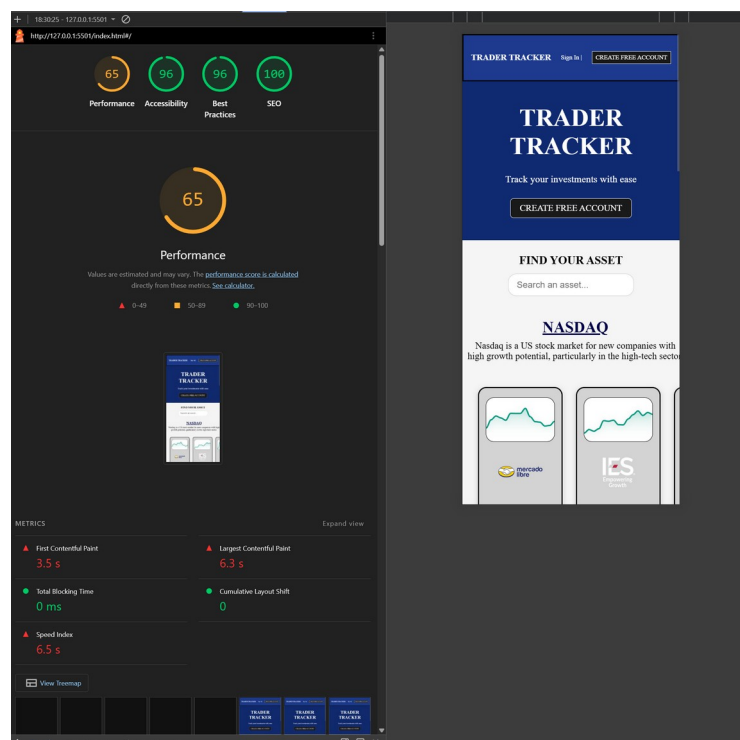
Bonnes pratiques : 96 / 100

SEO : 100 / 100

Le score Performance, plus bas, s'explique notamment par le First Contentful Paint (3,5 s) et le Largest Contentful Paint (6,3 s), liés au chargement des images d'actifs et aux appels à l'API au démarrage.

Les scores Accessibilité et Bonnes pratiques (96/100) seront portés à 100 en finalisant les derniers ajustements de contraste et d'attributs ARIA identifiés par l'audit.

Ces optimisations font partie des actions prévues avant la mise en production.



Audit WebAIM de contraste entre le texte et l'arrière-plan avec un résultat de 17.57:1 sur le Contrast Ratio.

The screenshot shows a web application titled "Contrast Checker". It has a breadcrumb trail: [Home](#) > [Resources](#) > Contrast Checker. The interface is divided into two main sections: "Foreground" and "Background".

Foreground:

- Hex Value: #000000
- Color Picker: A black color bar.
- Alpha: 1
- Lightness: A slider bar.

Background:

- Hex Value: #E5ECEF
- Color Picker: A light blue color bar.
- Lightness: A slider bar.

Contrast Ratio: 17.57:1 (displayed in a green box).

[permalink](#)

Normal Text

7.3 Sécurité et robustesse du front

Tout ce qui est dans le front-end est accessible et modifiable par l'utilisateur.

Il faut donc considérer le front-end comme non fiable pour la sécurité.

Les principales protections concernent les attaques XSS (Cross-Site Scripting),

la sécurisation des formulaires (validation côté client uniquement pour l'UX),

la protection contre le vol de tokens, et la transmission sécurisée des données vers le backend.

La sécurité réelle (comme la prévention des injections SQL) est assurée côté serveur via des requêtes préparées.

Concrètement, le risque XSS s'est matérialisé sur les champs de texte libre saisis par les utilisateurs (biographie d'analyste, commentaire de recommandation, nom de société) :

ces champs étant injectés dans le DOM via innerHTML pour leur affichage, un contenu malveillant (par exemple une balise <script>) aurait pu s'exécuter dans le navigateur de tout visiteur consultant la page concernée.

Ce risque a été neutralisé par une fonction d'échappement HTML (escapeHtml, dans utils/format.js), appliquée systématiquement avant l'insertion de tout contenu utilisateur dans le DOM.

Le détail de cette correction, ainsi que sa vérification par un test d'exploitation réel, est documenté en section 10.3 (Tests de sécurité, audit et correctifs appliqués).

8- IMPLEMENTATION DU BACK-END

8.1- Architecture et logique métier

Le backend est organisé autour d'un répertoire racine back contenant un point d'entrée index.js et un dossier src pour la logique métier :

Point d'entrée (index.js) :

Son rôle est d'initialiser l'application Express, de configurer les middlewares globaux (comme cors, la limite de débit de requêtes, les headers et express.json()), de définir le port d'écoute (via .env) et de lancer le serveur.

```
import express from "express"
import cors from "cors"
import rateLimit from "express-rate-limit"
import dotenv from 'dotenv'
import getConnection from "../src/db/connection.js"
import routes from "../src/routes/index.js"
import securityHeaders from "../src/middlewares/securityHeadersMiddleware.js"
import errorHandler from "../src/middlewares/errorHandler.js"

dotenv.config()

const app = express()
const PORT = process.env.APP_PORT || 3000

// SECURITY: protective HTTP headers (homemade, see securityHeadersMiddleware.js)
app.use(securityHeaders)

app.use(cors({
  origin: process.env.FRONTEND_URL
}))

app.use(express.json())
app.use(express.urlencoded({ extended: true }))

// SECURITY: limit login/register attempts to mitigate brute-force
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 20, // max 20 attempts per IP per window
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: "Too many attempts, please try again later." }
})
app.use("/auth", authLimiter)

app.use("/", routes)
app.use('/uploads', express.static('uploads'))

app.get('/', (req, res) => {
  res.send('Welcome to DWWMM Project!')
})

app.use(errorHandler)

async function startServer() {
  try {
    await getConnection()
    console.log("Database connection ON")

    app.listen(PORT, () => {
      console.log(`Listening at the port ${PORT}`)
    })
  } catch (error) {
    console.error("Database connection error:", error.message)
  }
}

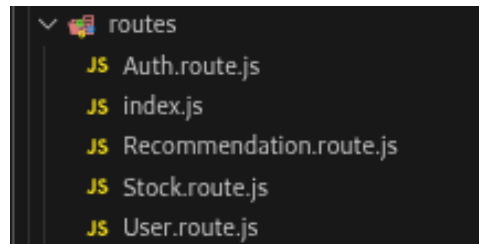
startServer()
```

Le dossier src/ :

Il centralise le code source découpé en sous-dossiers thématiques : routes, controllers, models, middleware, services, utils, scripts, data et db.

L'architecture utilise un système de routage organisé en « aiguillage » :

Un routeur d'index (src/routes/index.js) centralise toutes les branches de l'API.



Ce routeur redirige ensuite les requêtes vers des sous-routeurs spécifiques par entité (ex: user.routes.js, stock.routes.js).

back/src/routes/index.js

```
import express from "express"
import userRouter from "./User.route.js"
import authRouter from "./Auth.route.js"
import stockRouter from "./Stock.route.js"
import RecommendationRouter from "./Recommendation.route.js"

const router = express.Router()

router.use("/auth", authRouter)

router.use("/users", userRouter)

router.use("/assets", stockRouter)

router.use("/recommendations", RecommendationRouter)

export default router
```

Le fichier de route se contente de définir le chemin et le verbe HTTP (GET, POST, PUT, DELETE), puis appelle une fonction dédiée dans le contrôleur.

Extrait de back/src/routes/User.route.js

```
import express from "express"
import upload from "../services/multerConfig.js"
import UserController from "../controllers/UserController.js"
import AuthMiddleware from "../middlewares/authMiddleware.js"
import AssetMiddleware from "../middlewares/assetMiddleware.js"

const userRouter = express.Router()

// public
userRouter.get("/analysts", UserController.getAnalysts)

userRouter.get("/analysts/by-type", UserController.getAnalystsByType)

userRouter.get("/analysts/:id", UserController.getAnalystsById)
```

La Logique Métier dans les Contrôleurs :

C'est ici que réside la logique de traitement des requêtes HTTP:

Extraction et validation : Le contrôleur récupère et extrait les données du corps de la requête (req.body) ou des paramètres d'URL (req.params) et s'assure de leur validité avant de poursuivre le traitement.

Traitement : Il appelle les méthodes du modèle pour interagir avec les données.

Réponse HTTP : Il est responsable de renvoyer le code de statut approprié au frontend (ex: 200 pour une récupération réussie, 400 pour des données manquantes, 404 pour une ressource non trouvée).

En cas d'erreur celle-ci est transmise au middleware de gestion d'erreurs centralisé via next(error) qui s'appuie sur l'utilitaire AppError.js pour formater une réponse d'erreur cohérente.

back/src/controllers/UserController.js → getAnalysts()

```
async function getAnalysts(req, res, next) {
  try {
    const limit = Math.max(1, Number.parseInt(req.query.limit ?? 5, 10))
    const offset = Math.max(0, Number.parseInt(req.query.offset ?? 0, 10))

    const data = await UserModel.getAllAnalysts(limit, offset)

    return res.status(200).json(data);
  } catch (error) {
    next(error)
  }
}
```

Implémentation de la class `AppError` qui est instanciée dans les controllers.

`back/src/utls/AppError.js`

A code editor window with a light gray background and three colored window control buttons (red, yellow, green) in the top left corner. It contains the following JavaScript code:

```
export default class AppError extends Error {  
  constructor(message, statusCode = 400) {  
    super(message)  
    this.statusCode = statusCode  
    this.name = "AppError"  
  }  
}
```

une classe d'erreur personnalisée qui étend la classe native `Error` de Javascript.

Sans cette distinction, le backend renverrait uniquement un code 500.

8.2- Recommandations et autorisation par spécialisation

Le cœur métier de l'application repose sur la publication de recommandations (BUY / SELL / HOLD) par les analystes sur les actifs financiers.

Cette fonctionnalité illustre la mise en place d'une autorisation fine, au-delà d'un simple contrôle de rôle :

un analyste ne peut recommander que les actifs correspondant à sa spécialisation déclarée à l'inscription (ex : un analyste spécialisé Forex ne peut pas publier de recommandation sur une action Nasdaq).

La route POST /recommandations enchaîne trois middlewares avant d'atteindre le contrôleur :

back/src/routes/Recommendation.route.js

```
RecommendationRouter.post("/", AuthMiddleware(["analyst", "admin"]), AssetMiddleware(),  
SpecializationMiddleware(), RecommendationController.createRecommendation)
```

- AuthMiddleware

vérifie la présence et la validité du token JWT, et recharge l'utilisateur depuis la base de données pour s'assurer que son rôle réel (et non celui figé dans le token au moment de la connexion) est à jour ; le résultat est injecté dans req.user.

back/src/middlewares/authMiddleware.js

```
import UserModel from "../models/UserModel.js"  
import jwt from "jsonwebtoken"  
  
export default function AuthMiddleware(roles = []) {  
  return async function(req, res, next) {  
    const authHeader = req.header("Authorization")  
  
    const [prefix, token] = authHeader?.split(" ") || [null, undefined]  
  
    if (prefix !== "Bearer") {  
      return res.status(401).json({ error: "No Bearer token" })  
    }  
  
    if (!token) {  
      return res.status(401).json({ error: "You must be authenticated to access this resource" })  
    }  
  
    try {  
      const decoded = jwt.verify(token, process.env.JWT_SECRET)  
  
      if (!decoded?.id) {  
        return res.status(401).json({ error: "Invalid Payload" })  
      }  
  
      const user = await UserModel.getUsersById(decoded.id)  
  
      if (!user) {  
        return res.status(401).json({ error: "User not found" })  
      }  
  
      if (roles.length && !roles.includes(user.role)) {  
        return res.status(403).json({ error: "Permission denied, you are not authorized to access this resource" })  
      }  
  
      req.user = user  
  
      return next()  
    } catch (error) {  
      return res.status(401).json({ error: error.message })  
    }  
  }  
}
```


- AssetMiddleware

résout l'actif financier visé à partir du ticker transmis dans la requête, et l'injecte dans req.asset.

Si le ticker ne correspond à aucun actif connu, la requête est rejetée avant d'aller plus loin.

[back/src/middlewares/assetMiddleware.js](#)

```
import AssetModel from "../models/AssetModel.js"

export default function AssetMiddleware() {
  return async function (req, res, next) {
    try {
      //const user = req.user
      const ticker = req.body?.ticker || req.params?.ticker

      if (!ticker) {
        return res.status(400).json({ error: "ticker is required" })
      }

      const asset = await AssetModel.getAssetByTicker(ticker)

      if (!asset) {
        return res.status(404).json({ error: "Asset not found" })
      }

      req.asset = asset
      req.asset_id = asset.id
      //delete req.body.ticker

      return next()
    } catch (error) {
      console.error("ASSET MIDDLEWARE ERROR:", error)
      return res.status(500).json({ error: error.message })
    }
  }
}
```

- SpecializationMiddleware

compare le type de spécialisation de l'analyste (req.user.analyst_type_id) au type de l'actif visé (req.asset.asset_type_id). Un administrateur passe ce contrôle sans restriction ; un analyste dont la spécialisation ne correspond pas à l'actif reçoit une erreur 403 explicite ; tout autre rôle est rejeté.

[back/src/middlewares/specializationMiddleware.js](#)

```
export default function SpecializationMiddleware() {
  return async function (req, res, next) {
    try {
      const { role, analyst_type_id } = req.user //authMiddleware
      const { asset_type_id } = req.asset // assetMiddleware

      if (role === "admin") return next()

      if (role === "analyst") {
        if (analyst_type_id !== asset_type_id) {
          return res.status(403).json({ error: "Access denied: You can only recommend assets that match your specialization." })
        }
        return next()
      }
      return res.status(403).json({ error: "Access denied: Role not authorized." })
    } catch (error) {
      console.error("SpecializationMiddleware Error:", error)
      return res.status(500).json({ error: "Authorization check failed." })
    }
  }
}
```

Ce n'est qu'une fois ces trois vérifications passées que le contrôleur `RecommendationController.createRecommendation` reçoit la requête.

Il applique une dernière vérification métier (l'analyste doit avoir été validé par un administrateur — `analyst_verified` à 1 — pour pouvoir publier), puis transmet les données nettoyées (`sanitizeRecommendation`) au modèle, qui insère la recommandation en base via une requête préparée.

[back/src/controller/RecommendationController.js](#) → `createRecommendation`

```
async function createRecommendation(req, res, next) {
  try {
    if (!req.asset) {
      return res.status(400).json({ message: "Asset not resolved" })
    }

    if (req.user.role === "analyst" && Number(req.user.analyst_verified) !== 1) {
      return res.status(403).json({ message: "Your analyst account is pending validation. You cannot publish any recommendations." })
    }

    if (req.user.role === "analyst" && req.user.analyst_type_id !== req.asset.asset_type_id) {
      return res.status(403).json({ message: "You are not authorized to post on this type of asset." })
    }

    const sanitizedData = sanitizeRecommendation({
      status: req.body.status,
      comment: req.body.comment,
      asset_id: req.asset.id
    })

    const recommendation =
      await RecommendationModel.createRecommendations({
        ...sanitizedData,
        user_id: req.user.id
      })

    return res.status(201).json({ recommendation })
  } catch (error) {
    next(error)
  }
}
```

[back/src/utils/sanitizer.js](#) → `SanitizeRecommendation`

```
export function sanitizeRecommendation(data) {
  const { status, comment, asset_id } = data

  if (!status || !asset_id) {
    throw new AppError("Missing required fields")
  }

  const assetId = Number(asset_id)

  if (!Number.isInteger(assetId) || assetId <= 0) {
    throw new AppError("Invalid asset id")
  }

  const sanitizedData = {
    status: validateRecommendationStatus(status),
    comment: validateComment(comment),
    asset_id: assetId
  }

  return sanitizedData
}
```

back/src/models/RecommendationModel.js → createRecommendations

```
async function createRecommendations(data) {
  const db = getConnection()

  const { status, comment, asset_id, user_id } = data

  const sql = "INSERT INTO recommendations (status, comment, asset_id, user_id) VALUES (?, ?, ?, ?)"

  const [result] = await db.execute(sql, [status, comment, asset_id, user_id])

  return {
    id: result.insertId,
    status,
    comment,
    asset_id,
    user_id
  }
}
```

La modification et la suppression d'une recommandation (PUT et DELETE /recommendations/:id) suivent une logique d'autorisation complémentaire : le contrôleur vérifie que l'utilisateur authentifié est soit administrateur, soit l'auteur original de la recommandation (`existing.user_id === req.user.id`), avant d'autoriser l'action. Cette vérification protège contre une faille de type IDOR (Insecure Direct Object Reference), qui permettrait sinon à un analyste de modifier ou supprimer les recommandations publiées par un autre analyste simplement en changeant l'identifiant dans l'URL.

back/src/controller/recommendationController.js → Extrait de `updateRecommendation()`

```
// AUTHORIZATION
if (req.user.role !== "admin" && existing.user_id !== req.user.id) {
  return res.status(403).json({ message: "Update denied" })
}
```

Le contenu textuel des recommandations (champ comment) est par ailleurs systématiquement échappé à l'affichage côté front (fonction `escapeHtml`), pour empêcher qu'un commentaire malveillant ne s'exécute comme du code dans le navigateur des visiteurs consultant la page d'un actif (voir section 10 - Sécurité de l'application).

front/src/utills/format.js → `escapeHtml()`

```
export function escapeHtml(str) {
  if (str === null || str === undefined) return ""
  return String(str)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;")
}
```

8.3- Sécurité et intégrité des données

Middlewares de contrôle :

Ils interceptent la requête pour vérifier l'authentification via un jeton JWT (JSON Web Token) et s'assurer que l'utilisateur possède le rôle nécessaire (admin, jury, etc.) pour accéder à la ressource.

Hachage :

Les mots de passe ne sont jamais stockés en clair. Ils sont transformés en chaînes irréversibles via la bibliothèque bcrypt dans les fonctions utilitaires appelées dans le contrôleur avant l'insertion en base de données (voir 10 Sécurité de l'application)

9- ROUTES

Authentification (publique / session)

Méthode	Endpoint	Accès	Description
POST	/auth/register	Public	Inscription (upload photo + certif pour analyste)
POST	/auth/login	Public	Connexion, retourne un token JWT
GET	/users/me	Auth	Récupère les infos de l'utilisateur connecté
PUT	/users/me	Auth	Met à jour son profil (+ photo)

Actifs financiers (public)

Méthode	Endpoint	Accès	Description
GET	/assets/stocks	Public	Liste des actions (Nasdaq)
GET	/assets/forex	Public	Liste des paires de devises
GET	/assets/commodities	Public	Liste des matières premières
GET	/assets/home/stocks	Public	Données allégées pour la page d'accueil
GET	/assets/home/forex	Public	Données allégées pour la page d'accueil
GET	/assets/home/commodities	Public	Données allégées pour la page d'accueil
GET	/assets/brief/all	Public	Données minimales pour la liste complète
GET	/assets/details/:ticker	Public	Détails d'un actif (cours, historique)

Analystes (public)

Méthode	Endpoint	Accès	Description
GET	/users/analysts	Public	Liste des analystes validés
GET	/users/analysts/by-type	Public	Liste des analystes filtrés par spécialité
GET	/users/analysts/:id	Public	Profil détaillé d'un analyste

Espace Utilisateur (suivi d'actifs et d'analystes)

Méthode	Endpoint	Accès	Description
GET	/users/me/watchlist	User	Voir mes actifs suivis
GET	/users/me/watchlist-paginated	User	Voir mes actifs suivis (paginé)

Méthode	Endpoint	Accès	Description
POST	/users/me/follows	User	Suivre un actif
DELETE	/users/me/follows/:ticker	User	Ne plus suivre un actif
POST	/users/me/follows/users/:id	User	Suivre un analyste
DELETE	/users/me/follows/users/:id	User	Ne plus suivre un analyste
GET	/users/me/follows/users	User	Voir les analystes suivis
GET	/users/me/follows/users/:id/check	User	Vérifier si l'analyste est suivi

Espace Analyste (recommandations)

Méthode	Endpoint	Accès	Description
GET	/recommendations/me	Analyst, Admin	Voir mes recommandations
POST	/recommendations	Analyst, Admin	Créer une recommandation (BUY/SELL/HOLD)
PUT	/recommendations/:id	Analyst, Admin	Modifier une recommandation
DELETE	/recommendations/:id	Analyst, Admin	Supprimer une recommandation
GET	/recommendations	Public	Liste paginée de toutes les recommandations
GET	/recommendations/analyst/:analystId	Public	Recommandations d'un analyste donné

Administration

Méthode	Endpoint	Accès	Description
GET	/users	Admin	Liste paginée de tous les utilisateurs
GET	/users/pending-analysts	Admin	Liste des analystes en attente de validation
GET	/users/:id	Admin	Détails d'un utilisateur
PUT	/users/:id	Admin	Modifier un utilisateur (et valider un analyste via analyst_verified)
DELETE	/users/:id	Admin	Supprimer un utilisateur

10- SECURITE DE L'APPLICATION

10.1 Système d'authentification

Hachage des mots de passe (bcrypt)

Le hachage utilise un algorithme pour transformer une chaîne de caractères en un texte non reconnaissable, sans possibilité de retour en arrière (contrairement au chiffrement).

Le projet utilise la bibliothèque bcrypt avec un coût de hachage (salt rounds) de 10. Le mot de passe est haché avant l'insertion en base de données (fonction hashPassword), et comparé au hash stocké lors de la connexion (fonction comparePassword) — il n'est donc jamais stocké, transmis en interne, ni affiché en clair après sa saisie initiale.

back/src/utls/password.js : → password.js

```
import bcrypt from "bcrypt"

const SALT_ROUNDS = 10

async function hashPassword(password) {
  return bcrypt.hash(password, SALT_ROUNDS)
}

async function comparePassword(password, hashedPassword) {
  return bcrypt.compare(password, hashedPassword)
}

export { hashPassword, comparePassword }
```

Authentification via JWT (JSON Web Token)

Le JWT est la norme actuelle pour sécuriser les échanges entre le frontend et le backend sans maintenir d'état de session côté serveur.

Fonctionnement : à chaque connexion réussie (POST /auth/login) ou inscription (POST /auth/register), le serveur génère un jeton signé (fonction `generateToken`) contenant l'identifiant de l'utilisateur, son rôle, et sa spécialisation le cas échéant.

Ce jeton est signé avec une clé secrète (JWT_SECRET, isolée dans le fichier `.env`) et expire après 1 heure.

Ce jeton agit comme une « clé » que l'utilisateur doit présenter lors de chaque requête ultérieure pour prouver son identité, via l'en-tête Authorization (Bearer Token).

Il est stocké côté client dans le `localStorage`.

À chaque requête vers une route protégée, le serveur vérifie la signature du jeton (`jwt.verify`) — une signature invalide ou expirée entraîne un rejet immédiat (401) — puis recharge l'utilisateur correspondant depuis la base de données, garantissant que le rôle utilisé pour la suite de la requête est bien le rôle réel et actuel de l'utilisateur, et non une valeur potentiellement obsolète ou falsifiée contenue dans le jeton.

`back/src/services/authTokenService.js` : → `generateToken()`

```
import jwt from "jsonwebtoken"

function generateToken(user) {
  const token = jwt.sign(
    {id: user.id, role: user.role,
    analyst_type_id: user.analyst_type_id},
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  )

  return token
}

export default generateToken
```


10.2 Gestion de sessions, rôles et middlewares

Double Protection et Gestion des Rôles

L'application distingue plusieurs types d'utilisateurs : admin, user et analyst.

La sécurité doit impérativement être implémentée à deux niveaux :

- **Frontend** : un routeur SPA appelle la fonction `roleGuard` avant de charger une page privée, pour masquer les éléments visuels (comme le menu admin) selon le rôle stocké localement (voir section 7.1 - Implémentation du Front-End).

Cette protection reste cependant uniquement cosmétique.

- **Backend** : la protection réellement efficace.

Comme un utilisateur peut modifier manuellement son rôle dans son navigateur ou appeler directement l'API, le backend vérifie systématiquement le rôle réel de l'utilisateur (rechargé depuis la base de données, voir 10.1) via le middleware `AuthMiddleware` (rôles autorisés), placé sur chaque route sensible.

Pour les actions nécessitant une autorisation plus fine que le simple rôle (comme la publication de recommandations, limitée à la spécialisation de l'analyste), un middleware dédié supplémentaire (`SpecializationMiddleware`) est ajouté à la chaîne — voir section 8.2 - Implémentation du Back-End.

Sécurisation des Communications et de l'Environnement

- **HTTPS/SSL** : les données transitant sur un site en HTTPS sont chiffrées durant le transport, pour éviter qu'elles ne soient interceptées en clair sur le réseau.

- **Variables d'environnement (.env)** : les informations sensibles comme la clé secrète du JWT, les identifiants de la base de données ou la clé d'API externe sont stockées dans un fichier `.env`, qui ne doit jamais être partagé ou poussé sur un dépôt public.

- **CORS (Cross-Origin Resource Sharing)** : ce mécanisme permet de restreindre quelles URL (domaines) sont autorisées à interroger l'API backend, empêchant ainsi des sites tiers malveillants d'utiliser les ressources.

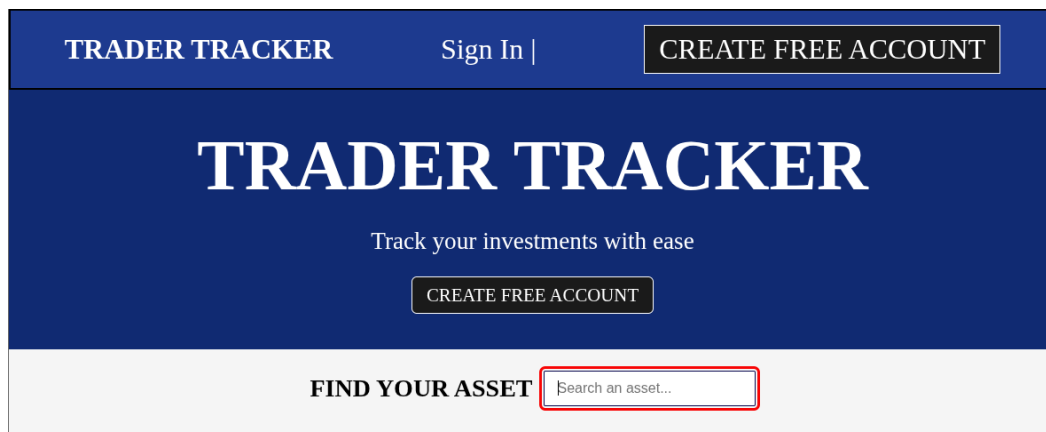
- **En-têtes de sécurité HTTP** : un middleware dédié (`securityHeadersMiddleware`) ajoute à chaque réponse du serveur des en-têtes protecteurs (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`), et supprime l'en-tête révélant la technologie backend utilisée (`X-Powered-By`).

- **Limitation des tentatives de connexion** : un middleware de rate limiting (`express-rate-limit`) restreint à 20 le nombre de tentatives autorisées par adresse IP sur une fenêtre de 15 minutes pour les routes `/auth`, afin de limiter les attaques par force brute.

11- ACCESSIBILITE

La démarche d'accessibilité du projet prend en compte les besoins des utilisateurs en situation de handicap, conformément aux recommandations du Référentiel Général d'Amélioration de l'Accessibilité (RGAA).

Sur le plan technique, cela se traduit notamment par l'utilisation d'une sémantique HTML5 appropriée (balises nav, main, section, footer plutôt que des div génériques), une hiérarchie de titres cohérente (un seul h1 par page, hiérarchie h2/h3 respectée), des attributs alt descriptifs sur toutes les images, et la mise en place d'un indicateur de focus visible.



11.1 Seo

Chaque page utilise une structure de titres sémantique (un seul h1 par page, hiérarchie h2/h3 cohérente) et des balises HTML5 appropriées (nav, main, section, footer) plutôt que des div génériques, ce qui facilite l'indexation par les moteurs de recherche.

Les images (logos d'actifs, photos d'analystes) disposent d'attributs alt descriptifs.

Les liens de navigation utilisent des libellés explicites plutôt que des formulations génériques comme « cliquez ici ».

11.2 Light house

Un audit Lighthouse (Chrome DevTools) a été réalisé sur la page d'accueil de l'application (voir 7.2 Esthétique, accessibilité et performances)

11.3 Rgpd

L'application collecte un minimum de données personnelles, strictement nécessaires à son fonctionnement : nom, email et mot de passe (haché) pour tout utilisateur, ainsi que société, biographie, photo de profil et document de certification pour les analystes.

Ces données ne sont jamais transmises à des tiers ni utilisées à des fins commerciales. Les mots de passe sont hachés via bcrypt et jamais stockés ou affichés en clair. L'utilisateur peut à tout moment mettre à jour ses informations personnelles depuis son espace.

Les variables sensibles (clé JWT, identifiants de base de données, clé d'API externe) sont isolées dans un fichier .env non versionné, conformément aux bonnes pratiques de protection des données recommandées par la CNIL et l'ANSSI.

12- TEST ET JEU D'ESSAI

12.1 Vérifications manuelles

Cinq scénarios de test ont été exécutés manuellement afin de vérifier le bon fonctionnement des parcours utilisateurs critiques de l'application.

Num test	Description	Prérequis	Données	Résultat attendu	Résultat obtenu
TEST001	En tant qu'utilisateur je souhaite accéder à l'application Trader Tracker	Aucun	Aucune	Le site est accessible	OK
TEST002	En tant qu'utilisateur inscrit je souhaite me connecter	Avoir un compte	login : testUser1@gmail.com , mdp : aA123456!	Utilisateur connecté	OK
TEST003	En tant qu'utilisateur inscrit je souhaite me connecter	Avoir un compte	login : fakeUser@mail.com , mdp : aA123456!	Le compte n'existe pas	OK
TEST004	En tant qu'utilisateur inscrit je souhaite suivre un actif financier	Avoir un compte et être connecté	login : testUser1@gmail.com , mdp : aA123456!	La carte de l'actif financier s'affiche sur la page du compte utilisateur	OK
TEST005	En tant qu'analyste je souhaite recommander un actif financier	Avoir un compte analyste et être connecté	login : testAnalyst1@gmail.com , mdp : aA123456!	La recommandation s'affiche sur la page de détail de l'actif, sur la page du compte analyste et sur le panel administrateur	OK

12.2 Tests automatisés

Une suite de tests automatisés est prévue via Jest afin de fiabiliser les évolutions futures de l'application et de détecter rapidement toute régression. Ces tests viendront compléter les vérifications manuelles réalisées jusqu'à présent (voir 12.1), en couvrant en priorité les fonctions critiques de l'application : authentification, gestion des rôles, et logique métier liée aux recommandations.

12.3 Jeu d'essai

Un fichier SEEDERS.md, accompagné d'un script SQL, est fourni à la racine du dossier back afin d'initialiser rapidement la base de données avec des données de test, sans passer par l'interface.

Il contient :

3 types d'actifs (Forex, Nasdaq, Comex)

3 comptes utilisateurs, un par rôle applicatif :

Rôle	Email	Mot de passe	Statut
admin	admin@trader-tracker.com	Admin123 !	Actif
user	user@trader-tracker.com	User123 !	Actif
analyst	analyst@trader-tracker.com	Analyst123 !	FALSE

Les mots de passe sont stockés en base de données sous forme de hash bcrypt. Les valeurs ci-dessus sont les mots de passe en clair à utiliser pour se connecter en test.

Cas particulier : le compte analyste

Le compte analyste est volontairement créé non vérifié (analyst_verified = FALSE), pour reproduire l'état réel d'un analyste qui vient de s'inscrire. Cela permet de tester le parcours complet :

- 1- Connexion avec le compte analyste : autorisée
- 2- Tentative de publication d'une recommandation : refusée (compte non vérifié)
- 3- Validation du compte depuis le panel admin (bouton Approve)
- 4- Publication d'une recommandation : autorisée après validation

Prérequis :

- la base de donnée trader_tracker doit déjà exister avec les tables créées (voir le fichier DATABASE.md à la racine du dossier back).
- Il est préférable de relancer le script après un TRUNCATE ou une réinitialisation de la base de données, car des données identiques sur les emails et les id provoqueront des erreurs du fait des contraintes UNIQUE.

13- DEPLOIEMENT DE L'APPLICATION

13.1 Préparation de l'environnement de production

L'application est composée de deux parties indépendantes : un serveur Node.js/Express (back) exposé via une API REST, et un front-end en JavaScript natif (HTML, CSS, JS) ne nécessitant aucune étape de build.

Côté base de données, un serveur MySQL doit être disponible. Le schéma est créé à partir des requêtes SQL documentées en section 6.1, puis les données de référence (types d'actifs, comptes de test) sont insérées via les scripts du dossier back/src/scripts.

13.2 Configuration

La configuration du back-end repose sur un fichier .env, jamais versionné, dont le modèle est fourni dans .env.example : paramètres de connexion à la base de données (DB_HOST, DB_USER, DB_PASSWORD, DB_DATABASE), port d'écoute de l'application (APP_PORT), clé secrète de signature des jetons JWT (JWT_SECRET) et clé d'accès à l'API externe de données financières (POLY_API_KEY, ex-Polygon.io).

Les dépendances du back sont installées via npm install, puis le serveur peut être lancé avec npm start (nodemon en développement).

Le middleware CORS est configuré pour n'autoriser que l'origine du front-end en production.

13.3 Mise en ligne et exploitation

Bien que l'application n'ait pas été mise en ligne à ce jour, une procédure de déploiement a été documentée afin de préparer une future mise en production.

Le front-end peut être déployé sur n'importe quelle plateforme d'hébergement web ou être directement servi par le serveur Express.

Le back-end est déployé sur un serveur ou une plateforme d'hébergement Node.js, avec les variables d'environnement de production renseignées côté serveur (jamais dans le code).

En production, le trafic transite en HTTPS afin de chiffrer les échanges entre le client et l'API, notamment lors de la transmission des identifiants et des jetons JWT.

Les dépendances utilisées dans ce projet sont toutes distribuées sous licences open source :

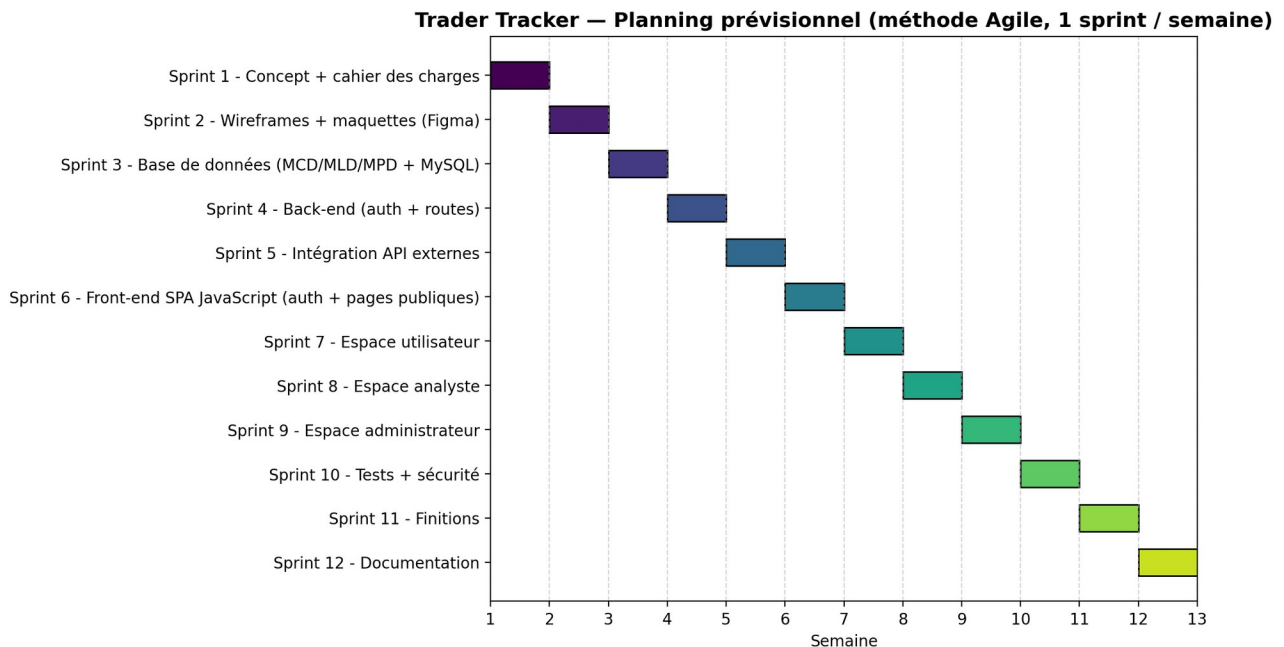
Express.js (MIT), bcrypt (MIT), jsonwebtoken (MIT), multer (MIT), mysql2 (MIT), dotenv (BSD-2-Clause), cors (MIT), express-rate-limit (MIT), nodemon (MIT), ApexCharts (MIT).

La librairie Massive API est soumise à sa propre licence commerciale.

Aucun actif protégé par le droit d'auteur (images, polices, sons) n'a été intégré sans vérification préalable de sa licence d'utilisation.

14- ROADMAP ET EVOLUTIONS FUTURES

14.1- Planning Prévisionnel



14.2- Évolutions futures possibles

- migration vers react ;
- migration vers l'offre payante de l'API Massive (anciennement Polygon.io), afin de bénéficier de données en temps réel et de lever les limitations du plan gratuit actuellement utilisé, ces données étant ensuite diffusées aux utilisateurs via WebSockets ;
- amélioration cosmétiques (design général, graphiques de prix) ;
- ajout d'un système de notation et de ranking des analyses ;
- possibilité pour les utilisateurs d'avoir un rôle plus actif sur les recommandations (commentaire & partage) ;
- page détail pour les recommandations ;
- conteneurisation de l'application avec Docker (back-end et base de données) ;
- gestion de logs ;
- CSRF (Cross Site Request Forgery) ;

CONCLUSION

Le projet Trader Tracker a permis de mettre en pratique l'ensemble des compétences du référentiel Développeur Web et Web Mobile, depuis la conception (users stories, MVP, modélisation MERISE) jusqu'à l'implémentation complète d'une application full-stack sécurisée.

Le choix d'un front-end en JavaScript natif et d'un routeur maison a permis de comprendre en profondeur le fonctionnement d'une Single Page Application, sans dépendre d'un framework.

Côté back-end, l'architecture MVC avec Express et MySQL a structuré une logique métier claire, avec une attention particulière portée à la sécurité :

hachage des mots de passe, authentification JWT, et vérification systématique des rôles côté serveur.

La gestion des quatre profils utilisateurs (visiteur, utilisateur, analyste, administrateur), le système d'authentification JWT couplé à un routeur SPA protégeant dynamiquement les routes selon le rôle (RoleGuard), ainsi que la publication de recommandations soumise à une autorisation fine par spécialisation, ont été des points clés du projet.

Ce projet a été une réelle source de motivation et confirme mon souhait de poursuivre mes études dans ce domaine, afin d'approfondir mes compétences en développement full-stack et d'aller vers des architectures toujours plus robustes et sécurisées.

ANNEXE

STACK TECHNIQUE ET MODULES APPLICATIFS

